Apache Pig: Web Server Log Analysis

Objective:

• Process raw web server access log data using Apache Pig.

• Load the access log into a Pig relation.

• Extract the client IP address and response bytes.

• Group records according to client IP address.

• Calculate the total bandwidth consumed by each IP address.

• Execute the Pig Latin program using the local/standalone environment.

• Store and process the data using Hadoop/HDFS when required.

Scenario:

A web hosting provider stores raw web server access logs. The operations team needs to analyze these logs to determine how much data has been transferred to different client IP addresses. Each log record contains information such as client IP address, date and time, HTTP request, HTTP response status, and number of response bytes transferred.

Problem Statement:

Write an Apache Pig program that:

• Reads an access log file.

• Extracts the client IP address.

• Extracts the response bytes from each record.

• Groups records belonging to the same IP address.

• Calculates the total number of bytes transferred for each IP address.

Mathematically: Total Bandwidth for an IP = Sum of response bytes for all requests from that IP.

Software and Tools Used:

Tool	Purpose
Apache Pig	Data flow scripting and analysis using Pig Latin.
Hadoop 3.5.0	Distributed processing framework.
HDFS	Distributed storage for the input log file.
YARN	Resource management and execution of Hadoop jobs.
WSL2 / Ubuntu	Linux environment used to run Hadoop and Pig.
Java	Required for running Hadoop.

Input File: access.log

The input file contains 8 Apache-style web server access log records. Example records:

```
192.168.1.10 - - [10/Jan/2026:10:00:00] "GET /index.html HTTP/1.1" 200 500
192.168.1.20 - - [10/Jan/2026:10:01:00] "GET /home.html HTTP/1.1" 200 1200
192.168.1.10 - - [10/Jan/2026:10:02:00] "GET /about.html HTTP/1.1" 200 800
192.168.1.30 - - [10/Jan/2026:10:03:00] "GET /login HTTP/1.1" 200 1000
192.168.1.20 - - [10/Jan/2026:10:04:00] "GET /products HTTP/1.1" 200 300
192.168.1.10 - - [10/Jan/2026:10:05:00] "GET /contact HTTP/1.1" 200 700
192.168.1.30 - - [10/Jan/2026:10:06:00] "GET /image.jpg HTTP/1.1" 200 400
192.168.1.20 - - [10/Jan/2026:10:07:00] "GET /data HTTP/1.1" 200 600
```

Here, the first field is the client IP address and the last field is the response bytes.

Apache Pig Working:

Step 1: Input — The access.log file contains web server requests.

Step 2: LOAD — Pig loads each line of the access log into a relation.

Step 3: FOREACH — Pig splits each record and extracts the IP address and response bytes.

Step 4: GROUP — Pig groups all byte values having the same IP address.

Step 5: SUM — Pig calculates the total response bytes for each IP address.

Step 6: DUMP/STORE — Pig displays or stores the final result.

Pig Latin Program: bandwidth.pig

```
logs = LOAD 'access.log' USING TextLoader() AS (line:chararray);

records = FOREACH logs GENERATE
          FLATTEN(STRSPLIT(line, ' ')) AS fields;

data = FOREACH records GENERATE
       fields.$0 AS ip:chararray,
       (long)fields.$9 AS bytes:long;

grouped = GROUP data BY ip;

result = FOREACH grouped GENERATE
         group AS ip,
         COUNT(data) AS requests,
         SUM(data.bytes) AS total_bandwidth;

DUMP result;
```

Note:

The field position can vary depending on the exact log format. For the Apache-style sample above, the response-byte field is the last whitespace-separated field. If the input is changed, the Pig field extraction should be adjusted accordingly.

Map/Reduce-style Processing in Pig:

Pig internally converts Pig Latin data-flow operations into execution jobs. The logical flow is:

```
Input access.log
      ↓
LOAD
      ↓
FOREACH / field extraction
      ↓
GROUP BY IP
      ↓
SUM(response bytes)
      ↓
Final result
```

Example Grouping:

• 192.168.1.10 → [500, 800, 700]

• 192.168.1.20 → [1200, 300, 600]

• 192.168.1.30 → [1000, 400]

Example Calculation:

• 192.168.1.10 → 500 + 800 + 700 = 2000 bytes

• 192.168.1.20 → 1200 + 300 + 600 = 2100 bytes

• 192.168.1.30 → 1000 + 400 = 1400 bytes

Local / Standalone Execution:

```
pig -x local bandwidth.pig
```

The local mode is useful for testing the Pig Latin script before running it with Hadoop.

Hadoop / HDFS Execution:

```
hdfs dfs -mkdir -p /input
hdfs dfs -put -f access.log /input/
hdfs dfs -ls -h /input
pig -x mapreduce bandwidth.pig
```

In Hadoop mode, the input file is stored in HDFS and Pig executes the data-flow operations using the Hadoop processing environment.

Expected Output:

```
(192.168.1.10,3,2000)
(192.168.1.20,3,2100)
(192.168.1.30,2,1400)
```

IP ADDRESS	REQUESTS	TOTAL BANDWIDTH
192.168.1.10	3	2000 bytes
192.168.1.20	3	2100 bytes
192.168.1.30	2	1400 bytes

Total Bandwidth:

2000 + 2100 + 1400 = 5500 bytes

Advantages of Apache Pig:

• Provides a simple data-flow language called Pig Latin.

• Reduces the need to write low-level MapReduce programs.

• Supports operations such as LOAD, FILTER, FOREACH, GROUP and JOIN.

• Can process large datasets using Hadoop.

• Useful for data preparation and analysis.

Conclusion:

The web server bandwidth analysis was implemented using Apache Pig. The Pig Latin program loads the access log, extracts the client IP address and response bytes, groups records by IP address, and calculates the total bandwidth transferred by each IP. For the sample data, the totals are 2000 bytes for 192.168.1.10, 2100 bytes for 192.168.1.20, and 1400 bytes for 192.168.1.30. The same data-flow approach can be tested locally and executed in a Hadoop/HDFS environment.

Subject: Data Engineering & MLOps

Topic: Apache Pig – Web Server Log Bandwidth Analysis